Operators and Expressions

✡ Objective

— To understand different operators available in C language

— To understand how expressions are evaluated.

✡ Operators

— What is operator? What is operand?

3 + 7 — operand, operand, operator

+7 −8 — unary operator

— Types of operators (Based on number of operands)

- Unary — operates on one operand (+, −, !, ++, --)
- Binary — " " two " (+, −, *, &&, <...)
- Ternary — " " three " (?:)

— Other classification (Based on what operator does)

1. Arithmetic operators (+, −, *, /, %)
2. Relational operators (>, >=, <, <=, ==, !=)
3. Logical operators (&&, ||, !)
4. Assignment operators (=, +=, -=, *=, /=, %=)
5. Increment and decrement operators (++, --)
6. Conditional operator (ternary operator) ?:
7. Bitwise operators
8. special operators.

3+7

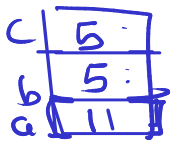
3>7 (0) false

(3<7) (1) true

a	b	a&&b	a||b	!a	!b
0	0	0	0	1	1
0	-17	0	1	1	0
-1	0	0	1	0	1
1	1	1	1	0	0

true

```
int a, b, c;
a = 5;
b = c = a;
```



② ①

= (5 + 6)

`a += 5;` // `a = a + 5;`

`7 += b;` / `7 = b;` Invalid. LHS must be variable

5. Increment and decrement operators

```
int num = 7;
```

post-increment → `num++;` // `num = num + 1;` | num = 8

pre-increment `++num;` // `num = num + 1;` | num = 9

(++) - one operator

```
int num2;
```

post-increment `num2 = num++;` // `num2 = num; num += 1;` | num = 10, num2 = 9

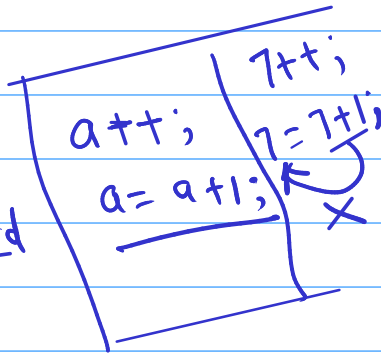
assign and then increment

→ `num2 = ++num;` // `num += 1; num2 = num;` | num = 11, num2 = 11

increment first and then assign

⇒ Try on your own

① `7++;`
`++7;`
`int x = 8++;`
} Valid? Not valid



② `float f = 9.7;`
`f++;`
`++f;`
} Valid? Yes

```
int m = 7, n;
```

pre-decrement → `n = --m;` // m = 6, n = 6 | `m -= 1; n = m;`

post-decrement → `n = m--;` // m = 5, n = 6 | `n = m; m -= 1;`

6. Conditional operator (Ternary operator)

```
int a;
7 > 5 ? a = 1 : a = 2;
```

operand | operand (if true) | operand (if false)

one operator

a would be 1.

```
int a, b, c;
a = 3, b = 5, c = 7;
a > b ? a-- : b--;
```

3 > 5

a = 3, b = 4, c = 7

```
int r = 9, s = 12, t;
t = r > s ? r : s;
```

12

finding maximum of r and s and storing it in t.

r = 9
s = 12
t = 12

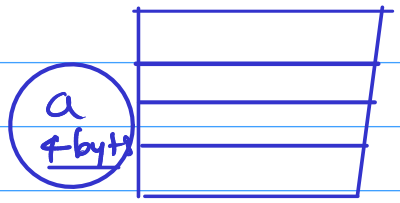
```
t = r > s ? s : r;
t = r < s ? r : s;
```

smaller of r and s is stored in t.

7. Bitwise operators

1. Bitwise logical operators ✓
2. Bitwise shift "
3. one's complement operator

1. Bitwise logical operators




```
int a=7, b=0;
```

Logical operators:

```
a && b   // 0
a || b   // 1
```

Bitwise logical operators:

```
a & b    // 0
a | b    // 7
```

a = 0 . . . . . 0111 (29)
b = 0 0000
0 0111

```
int p=10, q=20, r;
```

00 0 1 0 1 0
00 1 0 1 0 0
00 1 1 1 1 0
128 64 32 16 8 4 2 1
16 + 8 + 4 + 2 = 30

r = p && q;	p=10	q=20	r=1
r = p || q;	p=10	q=20	r=1
r = p & q;	p=10	q=20	r=0
r = p | q;	p=10	q=20	r=30

① Bitwise and (&)
② Bitwise or (|)
③ Bitwise xor (^) exclusive or

one bit

		&	|	^
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

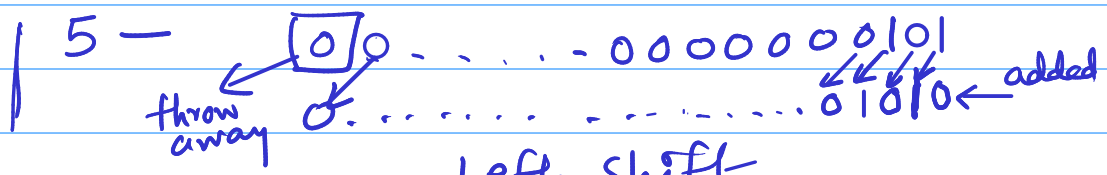
```
int a=12, b=17, c;

...000|100
...00|1001
-------
    |1101

c = a & b;  // a=12  b=17  c=0
c = a | b;  // a=12  b=17  c=29
c = a ^ b;  // a=12  b=17  c=29
```

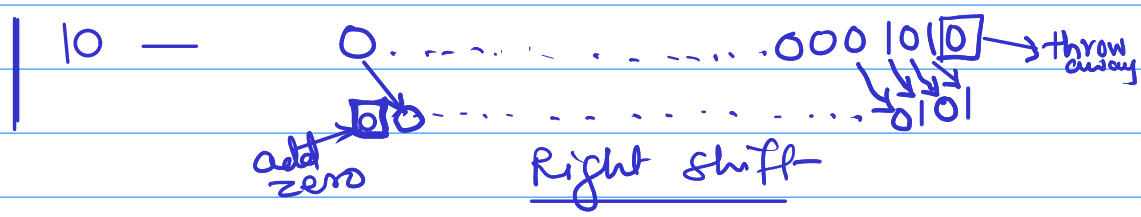
```
int x=7, y=29, z;

...000|111
...0|1110|1  ^
-------
  0|1010

z = x & y;  // x=7  y=29  z=5
z = x | y;  // x=7  y=29  z=31
z = x ^ y;  // x=7  y=29  z=26
```

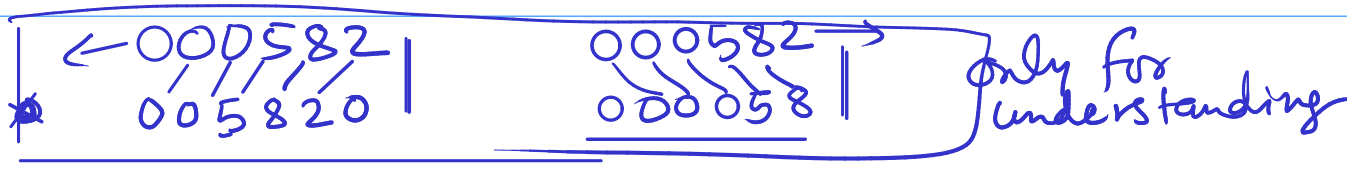
⇒ No bitwise operation on float or double.

```
Pnt t = 5 ^ 9;   ✓
t = 7 | 5;       ✓
8 & 4;   valid   But it has no impact.
```

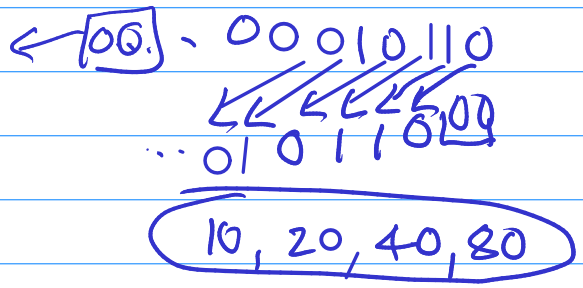
2. Bitwise shift operators







```
int x = 7;

x<<1;  // x = 7
x+1;   // x = 7

x = x << 1; // x = 14
x = x << 2; // x = 56
```




```
int z = 7;

z = z >> 1;  // z = 3

z = 98;

z = z >> 3; // z = 12
```

49, 24, 12

$z >> 4$ → $z/2^4$

3. one's complement operator → converts 0 to 1 and 1 to 0 in bits.

```
int x = 5;
x = ~x;
```

~ → complement operator